

49274 Space Robotics (Spring 2026)

Team Project: Exploring a Martian Cave

- **You will complete this project in teams of 1 to 3 students.**
 - Working individually is allowed but not recommended: the scope and assessment expectations are identical regardless of team size.
 - ✓ **Sign up your team** (even if an individual) on Canvas via the People > Project Team page.
- **Live demonstration:**
 - Due date: 9:30am, Monday 19 October 2026 (week 12) during the lab time. A schedule will be provided on the Canvas assignment page.
 - **25%** of the total mark of the subject.
 - ✓ In person with teaching staff during the normal lab time. This will involve a live demonstration of your methods working and a conversation about your design.
- **Report, video and code:**
 - Due date: 11:59pm, Friday 23 October 2026 (week 12) via Canvas
 - **15%** of the total mark of the subject.
 - ✓ One report, video and code submission per team. The report must include a **statement of individual contributions**. Students are also encouraged to complete a **peer review through Spark Plus**.
 - ✓ A written report that describes your approach and the results achieved.
 - ✓ A video that shows your methods working. Please upload this to YouTube, OneDrive or Google Drive and provide a link in your report that the marker can access.
 - ✓ Submit your completed source code to Canvas. Submitted code may be checked for similarity with other submissions and external sources. You remain responsible for all submitted work and must be able to explain how your code works.
- **Generative AI tools** may be used to support your learning and development in this assignment. Any use of generative AI must be briefly disclosed in your report, including what tools were used and how they assisted you. You remain responsible for checking, understanding and explaining all submitted work.
- **Further instructions for the demonstration, video, code, and report are provided at the end of this document.**

Exploring a Martian Cave

The planet Mars has many underground caves that are of particular interest to scientists. These caves were often formed by lava flows that have drained and left behind a stable cavity. There are many reasons why we want to explore these Martian caves:

- They contain **unique geological features** that give clues about the history of Mars,
- They possibly hold clues **about past or present life** on Mars, and
- They may be suitable for **future human habitation**, as they are protected from solar radiation, extreme temperatures, and dust storms.

Your team will develop software for a Mars rover that explores and understands a Martian cave environment. The robot needs to autonomously navigate an initially unknown environment to build a map and discover artefacts that are of interest to scientists. The robot is to explore and understand the environment autonomously, without any keyboard/mouse input from a human. A Gazebo simulation environment is provided, featuring a Martian cave-like environment and a robot with basic mapping and navigation capabilities – see the screenshot in Figure 1.

This project will involve designing, implementing and testing a robotic software system that draws on many concepts you have learnt in this subject: ROS with Python, decision making, computer vision, and SLAM.

You will use a combination of existing ROS packages and new code that you will write. Here are the main steps, with more details provided later in this document:

- Build a **vision dataset** of artefacts of interest for developing and testing a **computer vision model**
- **Detect** and **localise** artefacts of interest as the robot moves through the cave
- Autonomously **explore** and **navigate** the cave
- Perform close-range **inspection** of discovered artefacts
- **Integrate** perception and planning into autonomous robot behaviour
- Extend your system with **advanced capabilities** of your choice

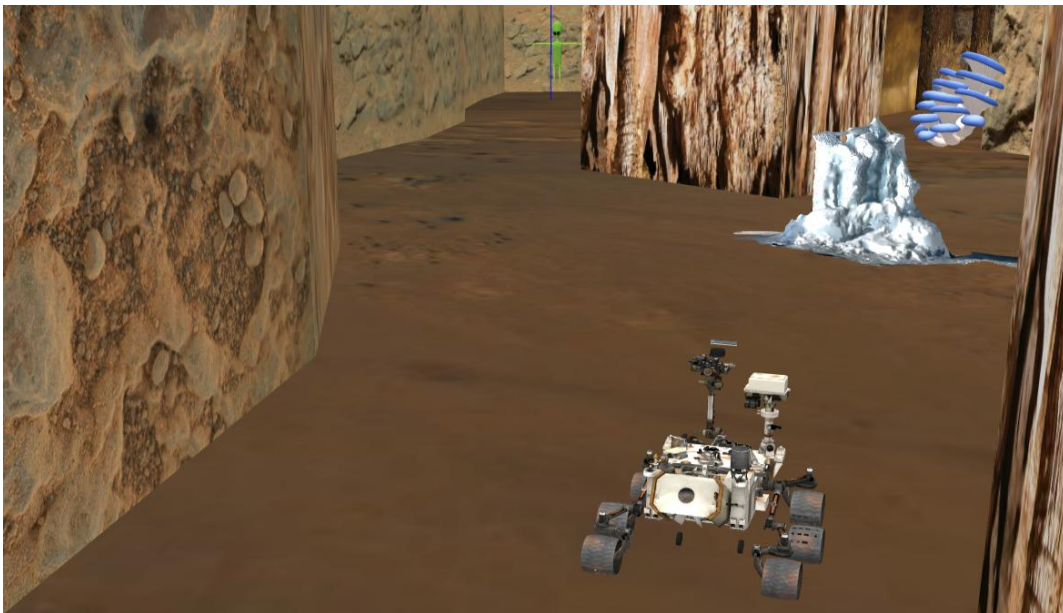


Figure 1: A screenshot of the simulation environment and robot you will be using. The Mars rover is navigating a simulated Martian cave world containing various artefacts of interest.

Getting Started

a) Installations

Begin by installing the Gazebo simulator (version Fortress), which is one of the most common 3D simulation engines used with ROS. To install Gazebo, follow the instructions detailed here:

https://gazebo.org/docs/fortress/install_ubuntu/

You will also use several existing ROS packages. Install these by running one command at a time:

```
sudo apt update
```

```
sudo apt install ros-humble-ros-ign-bridge ros-humble-ros-ign-gazebo
```

```
sudo apt install ros-humble-robot-localization
```

```
sudo apt install ros-humble-slam-toolbox ros-humble-navigation2 ros-humble-nav2-bringup
```

```
sudo apt install ros-humble-xacro
```

b) Download the template code

Next, download the provided template ROS package from the Canvas team project assignment page. Unzip this into your ROS workspace – it can be the same workspace that you used for the previous assignments.

Now build the package:

```
cd ~/ros_ws/  
  
colcon build --symlink-install --packages-select cave_explorer  
  
source install/setup.bash
```

These commands should run without errors. If you see any errors, address these first before proceeding.

c) Running your code

How you run your code may change a bit depending on your solutions; for example, you may create more launch files for your code. The following notes will start up the provided software package.

There are three launch files provided to you, which should be run together – one terminal for each launch file. You should take a closer look at the files in the `launch` directory to better understand what they provide.

The first launch file is `cave_explorer_startup.launch.py`, which launches the Gazebo simulator containing the robot and cave world, as well as the RViz visualisation window. Run this with:

```
ros2 launch cave_explorer cave_explorer_startup.launch.py
```

This may take a minute to load, particularly the first time you launch it. You should now see Gazebo and RViz windows similar to the following screenshots:



Figure 2: The Gazebo simulation window with the Martian cave world and robot in the bottom-right.

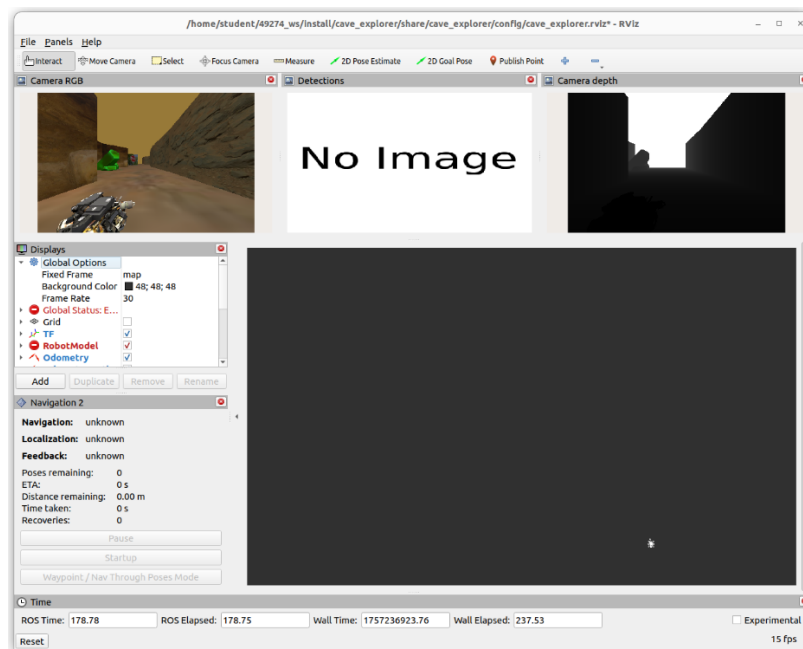


Figure 3: The RViz visualisation window when first launching `cave_explorer_startup.launch.py`, before the robot starts moving.

The second launch file is `cave_explorer_navigation.launch.py`, which launches some basic navigation capabilities including mapping and the Nav2 path planner pipeline for navigating around the environment. Run this with:

```
ros2 launch cave_explorer cave_explorer_navigation.launch.py
```

You should now see some more sensor data in RViz. This is the initial map that the robot will build over time as it starts moving. Your RViz window should now look like the following:

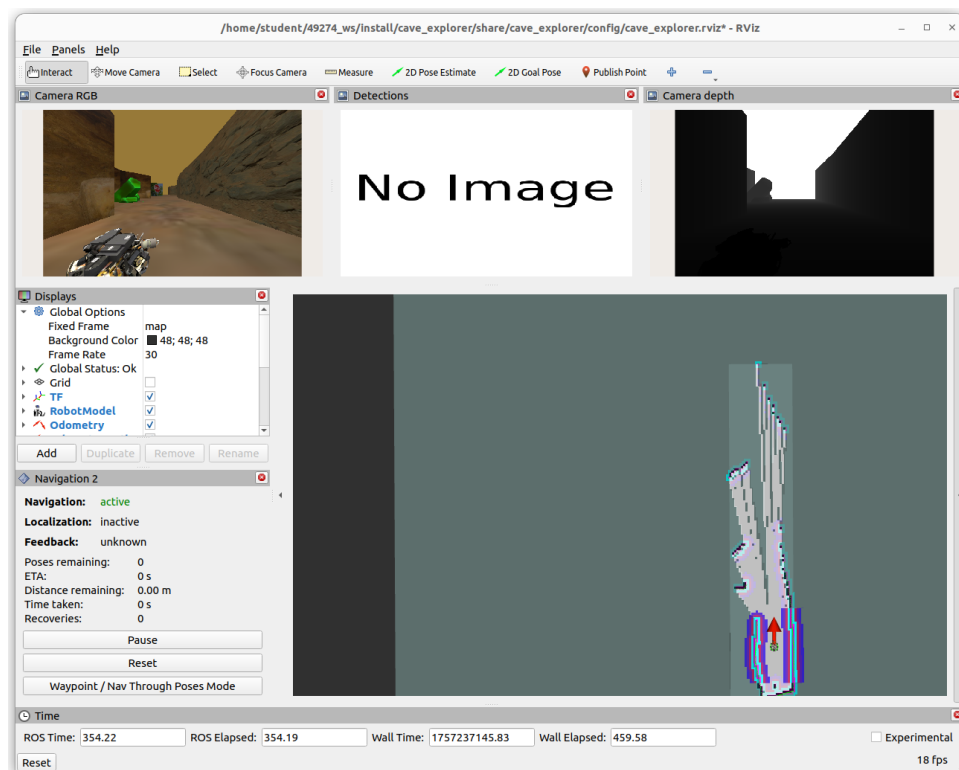


Figure 4: The RViz window should look like this after launching `cave_explorer_navigation.launch.py`

You can send goals to the Nav2 path planner by clicking the 2D Goal Pose button and then a location on the map. The robot will autonomously attempt to navigate to the goal while avoiding collisions. Try it now!

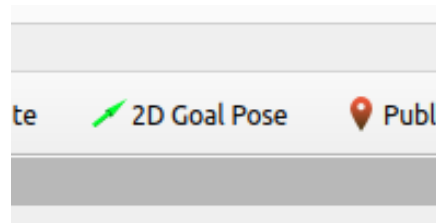


Figure 5: Click 2D Goal Pose and then select a location on the map to send the robot a navigation goal.

You can use this button when you want to send the robot to a particular location. However, for most of the project, the navigation goals will come from your software – making the robot fully autonomous!

This is what it should look like when navigating to a goal:

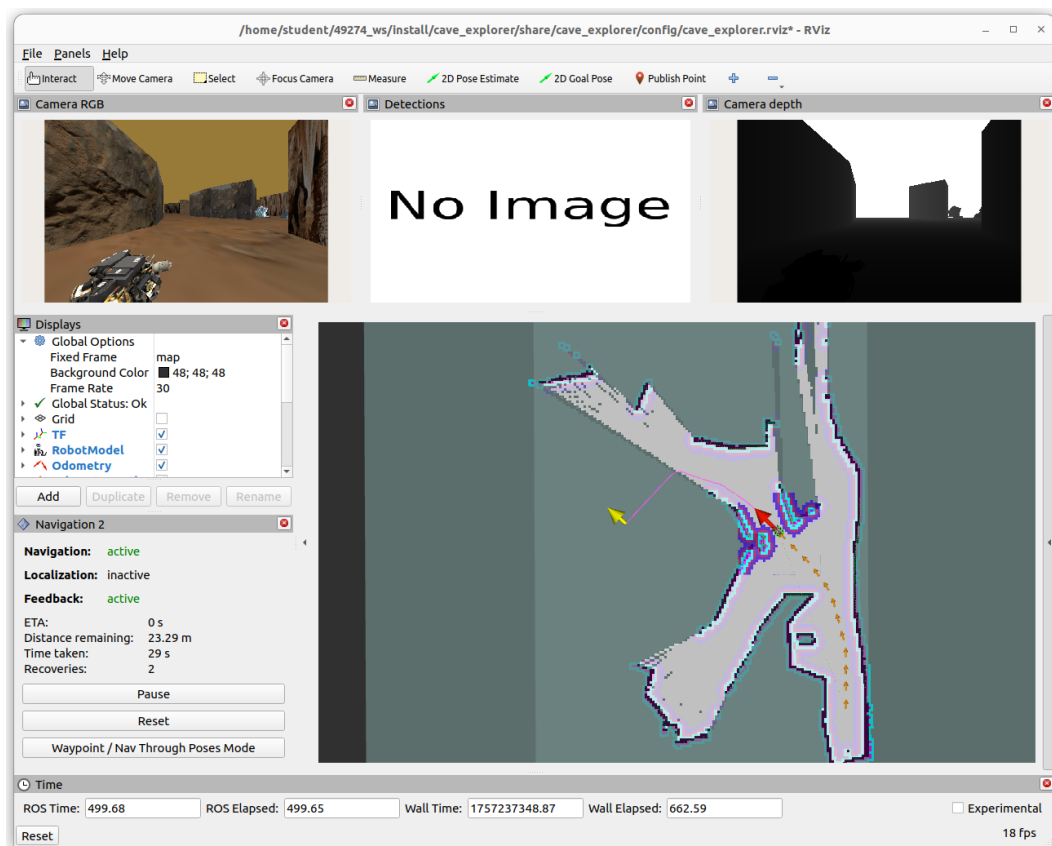


Figure 6: Attempting to navigate to a goal pose (yellow arrow).

In the bottom-left of RViz you should see the current status of the Nav2 path planner, which will indicate if something has gone wrong, such as an unreachable goal. You can reset with the Reset button.

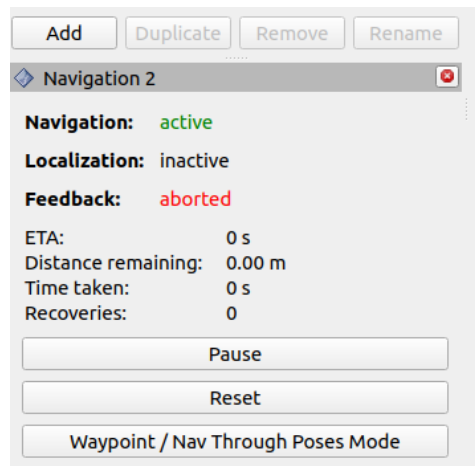


Figure 7: Status of the Nav2 navigation stack is shown in the bottom left of RViz.

The third and final launch file is `cave_explorer_autonomy.launch.py`, which launches the ROS node coded in `cave_explorer/cave_explorer.py`. This is where most of your project code should go.

Run this with:

```
ros2 launch cave_explorer cave_explorer_autonomy.launch.py
```

This ROS node already provides some basic decision making and computer vision capabilities to get you started. The provided code will automatically detect stop signs in the environment using the camera – you can see this in the top-middle of RViz. Additionally, the robot is initially programmed to go to a predefined location, return to the start location, and then move to random locations in the environment. Try it out and observe the behaviour!

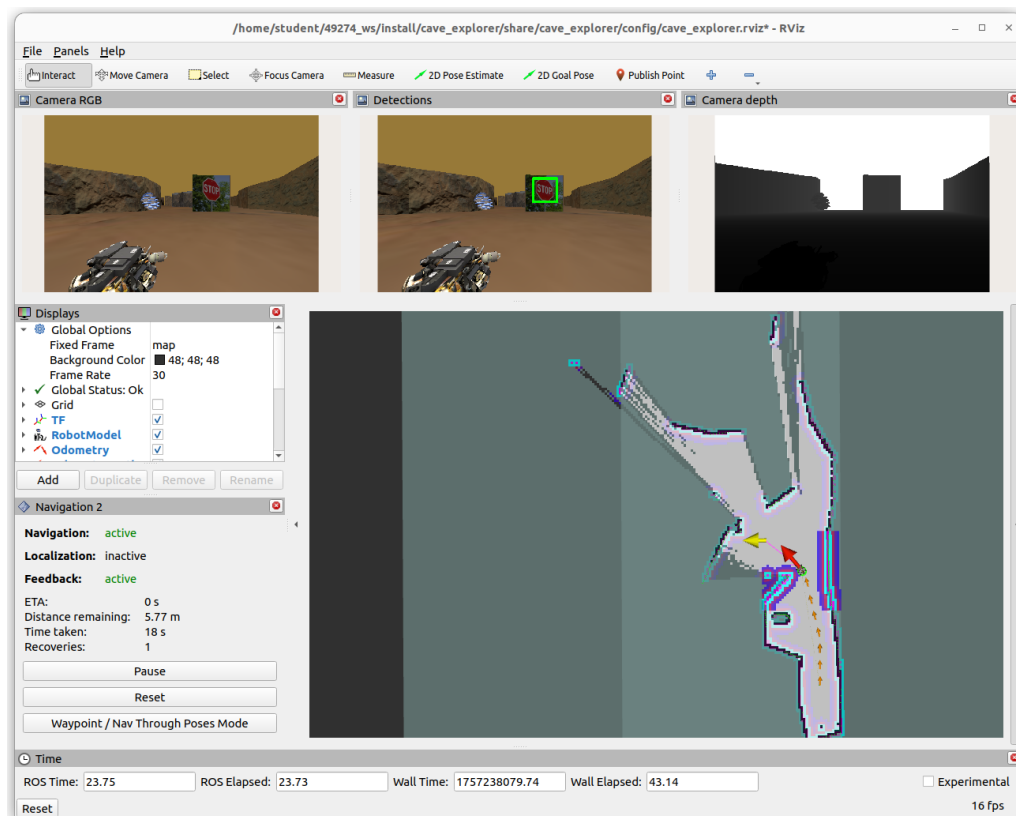


Figure 8: The RViz window when launching all three of the launch files and navigating towards the first goal. Basic computer vision (stop sign detection), mapping and navigation should already work. Can you work out what all of the different images and mapping colours mean? What sensors does the robot have?

Hint: You can shut down the second and third launch files during your testing without having to restart the Gazebo window, which can be slow to start up.

As part of your solution, you may wish to launch additional ROS nodes. These can be added to `cave_explorer_autonomy.launch.py` or you are welcome to create and run additional launch files.

Your Tasks

The goal of the project is to have a simulated Mars rover autonomously navigate and explore a cave-like environment and find, detect, and localise several visual artefacts of interest, in a fast and efficient manner.

Your mark for this project will be based on the demonstrated performance (e.g., accuracy, speed, efficiency), novelty (new and interesting approaches) and robustness (well tested) of your various methods for each task. Further marking criteria are provided in the table below.

Most tasks will require you to extend the provided Python code (`cave_explorer.py`). Some basic functionality is already provided to get you started. Unlike the previous assignments, you are welcome to change any part of the template code as you see fit, and may split your solution across multiple Python files if useful.

You are welcome to use external libraries and tools to support your implementation, provided these are appropriately acknowledged. You remain responsible for the design and integration of your solution, and must understand and be able to explain all submitted code.

Here is a summary of the tasks and marking scheme, which are explained further below. These task weightings apply across the Team Project as a whole, including both the live demonstration and the report/video/code submission. You don't necessarily need to complete the tasks in order, and team members may work on different tasks in parallel. I suggest the Perception and Planning tasks should be completed before attempting the Advanced tasks.

Tasks	Marking notes
Task 0: Understand and manage your project	No marks.
Perception 1: Collect an image dataset of the artefacts	35% of total mark. Full marks require all three tasks to be completed to a high standard.
Perception 2: Detect artefacts with computer vision	
Perception 3: Artefact localisation and display	
Planning 1: Autonomously explore the cave	35% of total mark. Full marks require all three tasks to be completed to a high standard.
Planning 2: Close-range inspection	
Planning 3: Behaviour switching	
Advanced 1: Robust perception	30% of total mark. Two high-quality Advanced tasks can earn full marks for this component. Partial or lower-quality attempts will receive proportionally fewer marks.
Advanced 2: Cave geometry analysis	
Advanced 3: Communication network deployment	
Advanced 4: Online roadmap construction	
Advanced 5: Science tour planning	
Advanced 6: Adaptive scientific sampling	

Task 0: Understand and manage your project

Before you begin coding, make sure your team has a clear plan for how you will approach the project. Take time to discuss the following questions together:

- Do you all fully understand the project goals? Is there anything you need to clarify with the teaching staff?
- Can you all successfully run the instructions listed above under “getting started”?
- How will you communicate and collaborate as a team?
- How will you share your code and report?
- Which tasks are you aiming to attempt and complete?
- Who will be responsible for each individual task?
- What deadlines should you set to keep the project on track?

Remember: Your final report must include a **Statement of Individual Contributions** (see the last page) and you are encouraged to **peer review each other’s contributions through Spark Plus**.

Good planning here will save you time later and help ensure your project runs smoothly.

Perception 1: Collect an image dataset of the artefacts

The goal of the perception tasks is to accurately and reliably **detect and localise artefacts** hidden in the cave. Some artefacts are visually distinct and relatively easy to detect, while others are more challenging. As a starting point, the provided template code includes a simple detector for “stop signs”.

Your first step is to build a **dataset of images** from the simulated robot that you will later use to train your computer vision model. Begin with a small dataset, and return to collect more if your model struggles with certain artefact types. Aim to include all artefact types, though you may find that some are easier to collect useful training examples for than others. Remember that **dataset quality is more important than raw size** – carefully chosen, varied examples will lead to better performance than thousands of near-identical images.

Your dataset should include:

- Multiple examples of each artefact type from different angles.
- Negative examples (images with no artefacts) to improve model robustness.

The template code already subscribes to the camera image stream. Extend it to save selected images to file (not every frame) for building your dataset.

It is up to you how you move the robot while collecting data:

- Use the provided random exploration behaviour in the template code,
- Specify waypoints, or
- Directly teleoperate the robot using:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

In your report: Include a few example images and list how many of each artefact type you collected.

Perception 2: Detect artefacts with computer vision

Building on Perception 1, your next task is to design and test a **computer vision model** for detecting the various artefacts hidden in the cave. A simple detector and example code are provided for detecting ‘stop signs’; your goal is to extend this capability to the other artefact types.



Figure 9: Example of the "stop sign" detector that is provided in the template code.

It is okay if your model does not reliably detect every artefact type. Some artefacts are intended to be challenging. Focus first on the types you believe are easier, and then attempt the more difficult ones.

You may choose which computer vision methods to use. The provided stop sign example is based on OpenCV (from this tutorial <https://www.geeksforgeeks.org/detect-an-object-with-opencv-python/>), which could be a useful starting point. You may also wish to try out:

- YOLO ROS: https://github.com/mgonzs13/yolo_ros
- PyTorch: https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html
- Google Teachable Machine: <https://teachablemachine.withgoogle.com/>

You are also welcome to use the **depth camera** provided in the simulator, which can be subscribed to in the same way as the RGB colour camera. Consider whether depth, colour, or a combination works best for the artefacts.

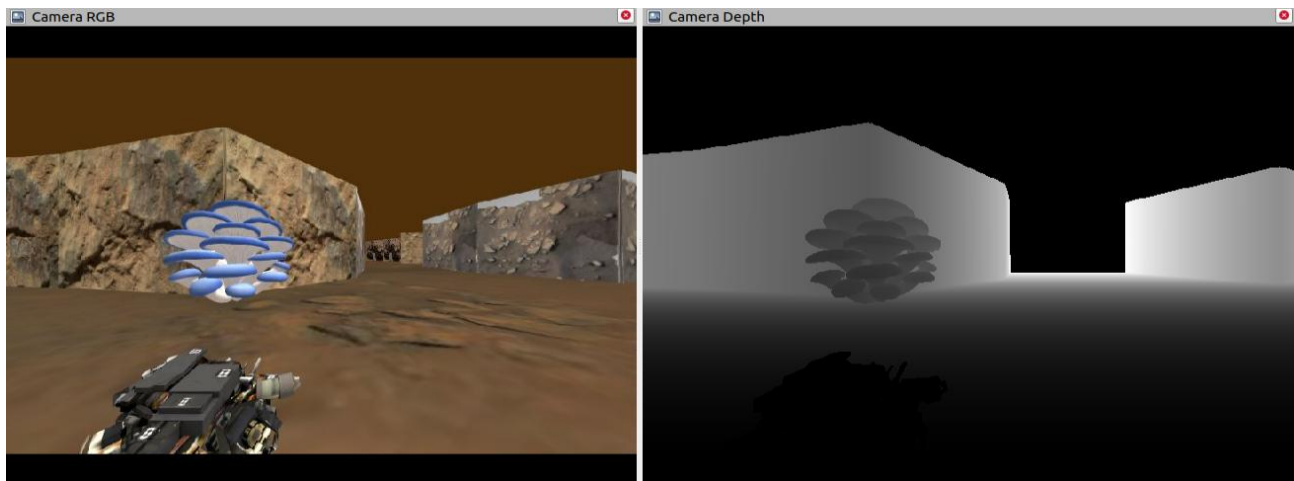


Figure 10: Colour and depth camera views of one of the artefacts. It is up to you whether you use colour or depth or both!

Important: Your detections must be shown by **annotating bounding boxes on the images**, as in the provided stop sign example. These annotated outputs should also be viewable in RViz (this is already set up for you).

In your report: Describe the methods and libraries you used, any challenges you encountered (including integration with ROS), and provide evidence of detection results (e.g., annotated images, RViz outputs).

Perception 3: Artefact localisation and display

By this stage, your robot should be able to reliably detect artefacts using the camera, displaying them as bounding boxes on the image stream. These bounding boxes indicate the direction of the artefact relative to the robot at the time of observation.

The next step is to estimate the **location of each artefact in the world map**. There are multiple ways to approach this. For example, you could:

- Estimate direction from the pixel coordinates of the detection, and
- Estimate distance using the depth camera.

Since multiple detections of the same artefact may yield slightly different position estimates, you should combine repeated observations using a suitable method (e.g., averaging or clustering).

The resulting location estimates must be displayed in RViz so they are clear to a human operator. Use **RViz markers** for this purpose (<http://wiki.ros.org/rviz/DisplayTypes/Marker>), as demonstrated in the template code for Assignment 1 and Assignment 2.

The functions `localise_artefact()` and `publish_artefact_markers()` are partly implemented for you. Currently, markers are displayed at the **robot's location** (see Figure below). Your task is to update the code so that the markers instead show your **estimated location** of the detected artefacts.

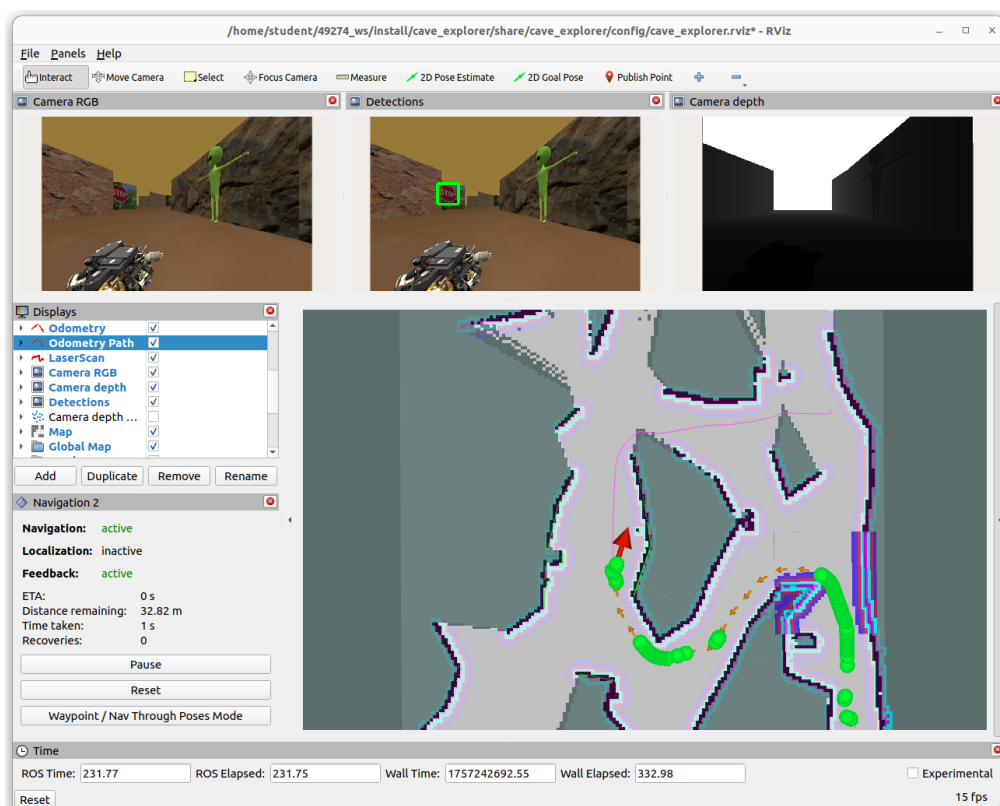


Figure 11: The green spheres along the trajectory are RViz markers. This screenshot shows the starting point for this task: You need to update the code to modify these markers so that they show the location of detected artefacts (not just the robot location).

In your report: Describe your method for estimating artefact positions. Explain how you handled multiple detections of the same artefact. Provide evidence of correct localisation (e.g. RViz screenshots showing artefacts marked in the cave map).

Planning 1: Autonomously explore the cave

Design and test an autonomous strategy for **efficiently exploring** an initially unknown cave. The robot should build a map while discovering new areas.

- The template code includes simple behaviours that move to random or pre-specified locations. Use this as a starting point to create your own `PlannerType` and add this to the `main_loop`.
- Review the **49274 Decision Making** seminar slides for approaches; a **frontier-based exploration** method is a strong starting point.

- Start **simple**, then add features incrementally to improve performance (e.g., better frontier selection, revisitation avoidance, path cost heuristics).
- You may draw inspiration from online resources, but you **must write your own code**.
- Add RViz markers/overlays (e.g., chosen frontier, candidate goals, path) to help visualise how your exploration is working.

In your report: Describe your exploration method and design choices, comment on how well it works, and outline additional features you would add with more time.

Planning 2: Close-range inspection

While exploring the cave, the robot will discover artefacts of interest. Initially you may only detect stop signs; after completing Perception 2, you should detect additional artefact types. In this task, when a chosen artefact type is detected, the robot must **navigate to a suitable close-range viewpoint for detailed inspection**. Pick **2 or 3 artefact types** that your detector handles reliably and focus on those.

Target behaviour (sequence):

1. The robot **autonomously explores** using your Planning 1 strategy.
2. On detecting one of the chosen artefact types, the robot **pauses exploration**.
3. The robot **generates an approach goal** near the detected artefact (e.g. set a standoff distance and viewing angle) and **navigates to it**, keeping the artefact in view.
4. For *Planning 2*, the behaviour **ends at close range** (Planning 3 will continue from here).

Implementation suggestions:

- Convert detections to map-frame goals (use Perception 3 localisation or an approximate bearing + depth estimate).
- Use a **safe standoff distance** and **heading alignment** so the camera faces the artefact.

In your report: Describe your approach, and note any difficulties and how you addressed them.

Planning 3: Behaviour switching

Extend the robot's behaviour to alternate between **exploration** (from Planning 1) and **close-range inspection** (from Planning 2). The robot should inspect newly detected artefacts of the chosen types while avoiding inspecting the same artefact more than once.

Target behaviour (sequence):

1. The robot **autonomously explores** using your Planning 1 strategy.
2. When a chosen artefact type is detected, the robot stops exploring and switches to the close-range inspection behaviour from Planning 2.
3. While approaching the artefact, use a **timeout or fallback**: if the artefact is lost, retry once; if still unsuccessful, abandon the attempt and return to exploration.
4. After a successful inspection, the robot records that artefact as visited and resumes exploration until the next new artefact is detected.

Implementation suggestions:

- Maintain a record of already-inspected artefacts to prevent duplicates.
- Use RViz markers to show which artefacts have been visited and which remain.

In your report: Describe how you approached this task and any difficulties you faced. It is okay if the switching behaviour does not work every time – this task may be more challenging than it first appears. If you had more time, how could you improve the robustness of your behaviour switching?

Advanced 1: Robust perception

In this task, extend your solution and analysis from Perception 1-3 for environments with **additional perceptual challenges** inspired by real Martian conditions. Examples include dust, poor lighting, low-quality cameras, dirty lenses, or motion blur.

You do **not** need to modify the Gazebo world or simulation plugins. Instead, **apply degrading visual effects** to the images received in `cave_explorer.py`, then feed these degraded images into your computer vision pipeline.

You must:

- Show the original and degraded image streams in RViz, alongside the detection outputs (as in Perception 2).
- Try at least two different types of visual degradation, with multiple levels of severity.
- Evaluate how these degradations affect the performance of your perception system.
- Improve your vision model or pipeline to address these challenges.
- Demonstrate and compare the performance before and after your improvements.

You are free to choose the types of degradation and how you improve the robustness of your system.

In your report: Describe the degradations you tested, explain how they affected perception performance, and describe the modifications you made to improve robustness. Provide evidence of improvement using suitable quantitative results and/or qualitative comparisons.

For examples of relatively simple visual effects in OpenCV, see:

https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

Advanced 2: Cave geometry analysis

Assignment 2 introduced distance transform maps for analysing obstacle clearance. In this task, extend that idea to analyse the geometry of the cave online as the robot explores.

Your goal is to automatically identify geometrically interesting regions of the cave, such as wide open areas that may be suitable for future human activity, and narrow passages that may be important for navigation or hazard assessment.

To complete this task, your system must:

- Compute or update a distance transform as the cave is explored.
- Analyse the distance transform to identify important geometric structures, such as wide regions, narrow passages, ridges, or saddle points.
- Identify and report useful geometric features during runtime, including at least:
 - The **widest accessible area** discovered so far, and
 - A **narrow passage** or bottleneck.
- Display the detected geometric features in RViz, overlaid on the current cave map and/or distance transform.
- Demonstrate that the identified features correspond sensibly to the geometry of the cave.

You are encouraged to use the **gradient/Jacobian of the distance transform**, or another suitable method, to detect and characterise these structures.

In your report: Describe how your method works, explain how you identify the different geometric features, and provide evidence of correctness using suitable screenshots, plots, or measurements.

Advanced 3: Communication network deployment

In this task, extend your system so that the robot maintains a communication link back to the starting location as it explores the cave. The robot can deploy communication relay nodes, with the assumption that two nodes can communicate if they are within a fixed distance and have line of sight.

You do **not** need to simulate radio communications or networking protocols. Treat this as a geometric planning problem: your goal is to decide **where relay nodes should be deployed** so that the robot remains connected to the existing network while using as few relays as possible.

To complete this task, your system must:

- Begin with a simple **greedy deployment strategy**, where the robot deploys a relay shortly before it would lose connection to the existing network.
- Extend this with a more advanced strategy that plans relay locations more efficiently.
- Maintain and update the communication network as the robot explores new areas.
- Display the relay nodes and communication links in RViz, overlaid on the cave map.
- Compare the performance of your strategies using suitable measures, such as the **number of relay nodes required**, explored area covered, or robustness of the communication chain.

You are free to choose how you model communication range, line of sight, and relay placement, provided these assumptions are clearly explained.

In your report: Describe your greedy and improved deployment strategies, explain the assumptions used for communication, compare their performance, and provide evidence of successful network deployment using screenshots and/or suitable metrics.

Advanced 4: Online roadmap construction

Assignment 2 introduced path planning over a navigation roadmap. In this task, you will extend that idea by constructing a roadmap **online as the robot explores the Martian cave**.

A roadmap provides a sparse, high-level representation of the environment that can be useful for navigation and for understanding the overall structure of the cave.

To complete this task, your system must:

- Construct the roadmap incrementally as the robot explores, adding new nodes and edges based on the cave regions observed so far.
- Begin with a simple strategy that adds nodes at the robot's current position at suitable intervals and connects nearby nodes where appropriate.
- Extend this approach by also adding nodes and edges in **observed but not yet visited free space**, so that the roadmap represents more than just the robot's travelled path.
- Ensure that roadmap connections are valid and do not pass through obstacles.
- Display the evolving roadmap in RViz using suitable markers. You may reuse or extend the Graph data structure and RViz visualisation methods from Assignment 2.

You are encouraged to consider how the placement and density of roadmap nodes affect how useful the roadmap is for representing the cave.

In your report: Explain how your roadmap is constructed and updated during exploration, describe the differences between your initial and improved approaches, and provide evidence of correctness using annotated maps, RViz visualisations, and/or examples of navigation using the constructed roadmap.

Advanced 5: Science tour planning

After exploring the cave, the robot may need to visit a set of scientifically or operationally important locations, such as detected artefacts, cave junctions, wide chambers, narrow passages, or other regions of interest.

In this task, you will plan and execute an efficient route that visits a set of important locations in the cave. This is closely related to the **Travelling Salesman Problem (TSP)** introduced in the 49274 Decision Making seminar.

To complete this task, your system must:

- Define a suitable set of around **8–12 meaningful locations** in the cave. These should have a clear purpose and should not be chosen arbitrarily.
- Determine the travel cost between locations using the robot's navigation map or planner, rather than simply using straight-line distance.
- Begin with a simple baseline strategy, such as visiting the locations in discovery order or nearest-neighbour order.
- Implement or integrate a **TSP-style optimisation method** to find a more efficient visiting order.
- Execute the resulting route with the robot.
- Display the selected locations and planned tour in RViz.
- Compare the optimised route with your baseline using a suitable measure such as total travel distance or time.

You may implement your own TSP solver or use an existing solver or library, provided this is clearly acknowledged and you can explain how it works and how you integrated it into your system.

In your report: Explain how and why you selected the locations, describe your baseline and optimised planning methods, and compare their performance. Provide evidence such as route lengths, execution results, or RViz screenshots.

Advanced 6: Adaptive scientific sampling

In this task, extend your autonomous exploration system to perform **adaptive scientific sampling** of a spatially varying quantity. For example, this could represent temperature, water or ice concentration, radiation level, mineral concentration, or another scientifically interesting measurement.

You will first define a spatial measurement field, then use measurements collected by the robot to estimate this unknown field and decide where the robot should sample next.

To complete this task, your system must:

- **Implement or source a suitable spatial measurement field** for a scientifically meaningful quantity. This could be a simple synthetic or made-up dataset that you create yourself, or data adapted from an existing source. The data does **not** need to realistically match the cave environment, provided it gives you a reasonable spatial field for demonstrating and evaluating adaptive sampling.
- Allow the robot to obtain a measurement of this quantity at its current location.
- Use the collected measurements to build and update a **Gaussian Process (GP)** model of the spatial field.
- Use the GP prediction and uncertainty to **adaptively choose new locations to sample**, rather than simply sampling randomly or at fixed locations.
- Display the estimated field, sampling locations, and/or prediction uncertainty using suitable visualisations.
- Compare your adaptive sampling strategy against a simple baseline, such as random sampling or regularly spaced sampling.

Your adaptive sampling algorithm must use **only measurements already collected by the robot**. It must not access the underlying ground-truth measurement field or its parameters when deciding where to sample

next. The ground truth may, however, be used afterwards to evaluate how accurately your method reconstructed the field.

You are free to choose how you generate or source the underlying measurement field. The aim of this task is **not** to develop an accurate physical model of a Martian cave; the measurement field simply needs to provide a meaningful spatially varying quantity that the robot can sample and estimate.

You are **not expected to implement Gaussian Process regression from scratch**. You may use an existing library such as scikit-learn's `GaussianProcessRegressor`, which provides both predictions and uncertainty estimates. The following scikit-learn resources may be useful:

- **Gaussian Processes guide:** https://scikit-learn.org/stable/modules/gaussian_process.html
- **Gaussian Processes regression: basic introductory example:**
https://scikit-learn.org/stable/auto_examples/gaussian_process/plot_gpr_noisy_targets.html

The introductory example is one-dimensional, but the same approach can be applied using the robot's (x, y) position as the GP input. It also demonstrates how to obtain predictive uncertainty, which can be used when deciding where the robot should sample next.

In your report: Describe your measurement field, Gaussian Process implementation, and adaptive sampling strategy. Explain how new sampling locations are selected, and provide evidence showing how the estimate improves as more measurements are collected. Compare the adaptive strategy with your chosen baseline using suitable plots, visualisations, or quantitative measures.

Your Demonstration

You will give a live, in-person demonstration of your code in Week 12. **All team members are expected to attend and participate.** You should demonstrate what you have achieved for all tasks attempted. Every team member should be able to explain the system, their own contributions, and the design and implementation decisions made by the team. Teaching staff will ask you questions and may ask you to demonstrate particular aspects of your solution.

Do not prepare slides for this demonstration—it is intended to be an interactive, live session. You may keep **backup videos** in case of technical issues, but the main focus should be on a **working live demo and conversation about your work**.

Your Video

You must also prepare a **screen recording video** (or multiple shorter videos) that demonstrates the tasks you completed. This should be in a similar style to your Assignment 1 and 2 videos:

- Aim for approximately 5 minutes total length (this is a guideline, not a strict limit).
- No audio or narration is required – descriptions go in your report and at the live demonstration.
- If you submit multiple short videos (e.g. one per task), make this clear in your report.
- Upload your video(s) to YouTube, OneDrive, or Google Drive, and include a link in your report. **Ensure that the marker has permission to view the video – inaccessible videos will not be marked.**

Your Code

You must also upload your code through Canvas along with your report.

- This may be a single Python file (e.g. `cave_explorer.py`) or a zip file containing all relevant files you created or modified.
- Do **not** upload large model files to Canvas.

Your Report

Your team will submit one report describing what you have achieved. I expect most reports will be approximately 10–20 pages, including figures. This is a guideline rather than a target: be concise in what you are trying to communicate, as longer reports will not necessarily score higher. Include important code snippets, figures, screenshots, graphs and plots where useful.

Your report should follow this structure:

- **Group member details:** Names and student numbers.
- **Generative-AI disclosure:** Briefly state which generative-AI tools, if any, were used and how they assisted your work. Write “No generative AI tools were used” if applicable.
- **Statement of individual contributions** (template provided below). This must list all tasks attempted/completed, who contributed, and how. If missing, marks will be penalised and you will need to explain the absence to the Subject Coordinator. Team members who disagree with the statement may contact the Subject Coordinator privately. Students are encouraged to complete a peer review through Spark Plus, accessible through a separate Canvas assignment page.
- **Project overview:** Summarise your overall approach and what the team achieved. Be accurate – misleading statements will be penalised.
- **Task sections:** One per task attempted. Describe your approach, how well it worked, and how it could be improved with more time. Include key code snippets and screenshots as appropriate. Address guiding questions from the task descriptions above.

- **Results:** Describe what you achieved, and include a link to your video (see above). Use pictures, graphs, and/or plots where relevant as evidence of your working system.
- **Reflection on teamwork:** Describe how your team worked together, what challenges you faced, and what you learned. You may wish to revisit some of the discussion points from Task 0 here. If you worked individually, briefly reflect instead on how you planned and managed the project.

Example statement of individual contributions

Task	Completed?	Team member	Contribution
Perception 1	Yes	Name 1	Code and organised data
Perception 2	Yes	Name 2 Name 1	Wrote the code Helped with design and testing
Perception 3	Attempted but not working	Name 2	Designed and wrote the code
Planning 1	Yes	Name 3 Name 1	Wrote the code Helped with design
Planning 2	Attempted but not working	Name 2 Name 3	Wrote the code Helped debugging
Planning 3	Not attempted		
...			
Report writing	Yes	All	Writing
Recorded videos	Yes	Name 1 Name 3	Perception Tasks Planning Tasks